

User Guide to `luaossl`, Comprehensive OpenSSL Module for Lua

William Ahern

Daurnimator

February 26, 2026

Contents

1 Dependencies

1.1 Operating Systems

`luaoss1` targets modern POSIX-conformant systems. A Windows port is feasible and patches welcome. Note however that the module employs the POSIX thread API, POSIX `dlopen`, and the non-POSIX `dladdr` interface to protect OpenSSL in threaded environments.

1.2 Libraries

1.2.1 Lua 5.1, 5.2, 5.3

`luaoss1` targets Lua 5.1 and above.

1.2.2 OpenSSL

`luaoss1` targets modern OpenSSL versions as installed on OS X, Linux, Solaris, OpenBSD, and similar platforms.

1.2.3 pthreads

Because it's not possible to detect threading use at runtime, or to *safely* and dynamically enable locking, this protection is builtin by default. At present the module only understands the POSIX threading API.

Linking Note that on some systems, such as NetBSD and FreeBSD, the loading application must be linked against pthreads (using `-lpthread` or `-pthread`). It is not enough for the `luaoss1` module to pull in the dependency at load time. In particular, if using the stock Lua interpreter, it must have been linked against pthreads at build time. Add the appropriate linker flag to MYLIBS in `lua-5.2.x/src/Makefile`.

1.2.4 libdl

In multithreaded environments the module will initialize OpenSSL mutexes if they've not already been initialized. If the mutexes are initialized then the module must pin itself in memory to prevent unloading by the Lua garbage collector. The module first uses the non-standard but widely supported `dladdr` routine to derive the module's load path, and then increments the reference count to the module using `dlopen`. This is the safest and most portable method that I'm aware of.

1.3 GNU Make

The Makefile requires GNU Make, usually installed as `gmake` on platforms other than Linux or OS X. The actual Makefile proxies to `GNUmakefile`. As long as `gmake` is installed on non-GNU systems you can invoke your system's `make`.

2 Installation

All the C modules are built into a single core C library. The core routines are then wrapped and extended through Lua modules. Because there several extant versions of Lua often used in parallel on the same system, there are individual targets to build and install for each supported Lua version. The targets `all` and `install` will attempt to build and install both Lua 5.1 and 5.2 modules.

Note that building and installation can be accomplished in a single step by simply invoking one of the install targets with all the necessary variables defined.

2.1 Building

There is no separate `./configure` step. System introspection occurs during compile-time. However, the “`configure`” make target can be used to cache the build environment so one needn’t continually use a long command-line invocation.

All the common GNU-style compiler variables are supported, including `CC`, `CPPFLAGS`, `CFLAGS`, `LDFLAGS`, and `SOFLAGS`. Note that you can specify the path to Lua 5.1, Lua 5.2, and Lua 5.3 include headers at the same time in `CPPFLAGS`; the build system will work things out to ensure the correct headers are loaded when compiling each version of the module.

2.1.1 Targets

`all`
Build modules for Lua 5.1 and 5.2.

`all5.1`
Build Lua 5.1 module.

`all5.2`
Build Lua 5.2 module.

`all5.3`
Build Lua 5.3 module.

2.2 Installing

All the common GNU-style installation path variables are supported, including `prefix`, `bindir`, `libdir`, `datadir`, `includedir`, and `DESTDIR`. These additional path variables are also allowed:

`lua51path`
Install path for Lua 5.1 modules, e.g. `$(prefix)/share/lua/5.1`

`lua51cpath`
Install path for Lua 5.1 C modules, e.g. `$(prefix)/lib/lua/5.1`

lua52path

Install path for Lua 5.2 modules, e.g. `$(prefix)/share/lua/5.2`

lua52cpath

Install path for Lua 5.2 C modules, e.g. `$(prefix)/lib/lua/5.2`

lua53path

Install path for Lua 5.3 modules, e.g. `$(prefix)/share/lua/5.3`

lua53cpath

Install path for Lua 5.3 C modules, e.g. `$(prefix)/lib/lua/5.3`

2.2.1 Targets

install

Install modules for Lua 5.1 and 5.2.

install5.1

Install Lua 5.1 module.

install5.2

Install Lua 5.2 module.

install5.3

Install Lua 5.3 module.

3 Usage

3.1 Modules

3.1.1 openssl

Binds various global interfaces, including version and build information.

`openssl[]`

Table of various compile-time constant values. See also `openssl.version`.

name	description
<code>SSLEAY_VERSION_NUMBER</code>	OpenSSL version as an integer.
<code>LIBRESSL_VERSION_NUMBER</code>	LibreSSL version as an integer.
<code>NO_[feature]</code>	If defined, signals that this installation of OpenSSL was compiled without [feature].

`openssl.version(info)`

Returns information about the run-time version of OpenSSL, as opposed to the compile-time version used to build the Lua module. If *info* is not specified, returns the version number as an integer. Otherwise, *info* may be one of the following constants.

name	description
<code>SSLEAY_VERSION</code>	OpenSSL version description as a string.
<code>SSLEAY_CFLAGS</code>	Description of compiler flags used to build OpenSSL as a string.
<code>SSLEAY_BUILT_ON</code>	Compilation date description as a string.
<code>SSLEAY_PLATFORM</code>	Platform description as a string.
<code>SSLEAY_DIR</code>	OpenSSL installation directory description as a string.

`openssl.extensionSupported(ext_type)`

Returns a boolean indicating if the extension *ext_type* is handled internally by OpenSSL.

3.1.2 openssl.bignum

`openssl.bignum` binds OpenSSL's libcrypto bignum library. It supports all the standard arithmetic operations. Regular number operands in a mixed arithmetic expression are upgraded as-if `bignum.new` was used explicitly. The `__tostring` metamethod generates a decimal encoded representation.

`bignum.new(number)`

Wraps the sign and integral part of *number* as a bignum object, discarding any fractional part.

`bignum.interpose(name, function)`

Add or interpose a bignum class method. Returns the previous method, if any.

3.1.3 openssl.pkey

`openssl.pkey` binds OpenSSL’s libcrypto public-private key library. The `__toString` metamethod generates a PEM encoded representation of the public key—excluding the private key.

`pkey.new(string[, format[, type[, curve]]])`

Initializes a new pkey object from the PEM- or DER-encoded key in *string*. *format* defaults to “*”, which means to automatically detect the input encoding. If *format* is explicitly “PEM” or “DER”, then only that decoding format is used.

If specified, *type* may be “public” or “private” to indicate loading a public or private key, respectively.

In the case of loading an EC key in a format that does not include the curve parameters, the *curve* parameter should indicate the appropriate curve name.

On failure throws an error.

`pkey.new{ ... }`

Generates a new pkey object according to the specified parameters.

field	type:default	description
.type	string:RSA	public key algorithm—“RSA”, “DSA”, “EC”, “DH”, or an internal OpenSSL identifier of a subclass of one of those basic types
.bits	number:1024	private key size
.exp	number:65537	RSA exponent
.generator	number:2	Diffie-Hellman generator
.dhparam	string	PEM encoded string with precomputed DH parameters
.curve	string:prime192v1	for elliptic curve keys, the OpenSSL string identifier of the curve

The DH parameters “dhparam” will be generated on the fly, “bits” wide. This is a slow process, and especially for larger sizes, you would precompute those; for example: “openssl dhparam -2 -out dh-2048.pem -outform PEM 2048”. Using the field “dhparam” overrides the “bits” field.

`pkey.interpose(name, function)`

Add or interpose a pkey class method. Returns the previous method, if any.

`pkey:type()`

Returns the OpenSSL string identifier for the type of key.

`pkey:setPublicKey(string[, format])`

Set the public key component to that described by the PEM- or DER-encoded public key in *string*. *format* is as described in `openssl.pkey.new`—“PEM”, “DER”, or “*” (default).

`pkey:setPrivateKey(string[, format])`

Set the private key component to that described by the PEM encoded private key in *string*. *format* is as described in `openssl.pkey.new`.

`pkey:sign(digest)`

Sign data which has been consumed by the specified `openssl.digest` *digest*. Digests and keys are not all interchangeable.

Returns the signature as an opaque binary string¹ on success, and throws an error otherwise.

`pkey:verify(signature, digest)`

Verify the string *signature* as signing the document consumed by `openssl.digest` *digest*. See the `:sign` method for constraints on the format and type of the parameters.

Returns true on success, false for properly formatted but invalid signatures, and throws an error otherwise. Because the structure of the signature is opaque and not susceptible to sanity checking before passing to OpenSSL, an application should always be prepared for an error to be thrown when verifying untrusted signatures. OpenSSL, of course, should be able to handle all malformed inputs. But the module does not attempt to differentiate local system errors from errors triggered by malformed signatures because the set of such errors may change in the future.

`pkey:toPEM(which[, which])`

Returns the PEM encoded string representation(s) of the specified key component. *which* must be one of “public”, “PublicKey”, “private”, or “PrivateKey”. For the two argument form, returns two values.

`pkey:derive(peer)`

Calculate the shared secret (e.g. DH/ECDH) derived from the key and the peer’s (usually public) key.

3.1.4 `openssl.x509.name`

Binds the X.509 distinguished name OpenSSL ASN.1 object, used for representing certificate subject and issuer names.

¹Elliptic curve signatures are two X.509 DER-encoded numbers, for example, while RSA signatures are encrypted DER structures.

`name.new()`

Returns an empty name object.

`name.interpose(name, function)`

Add or interpose a name class method. Returns the previous method, if any.

`name:add(type, value)`

Add a distinguished name component. *type* is the OpenSSL string identifier of the component type—short, long, or OID forms. *value* is the string value of the component. DN components are free-form, and are encoded raw.

`name:all()`

Returns a table array of the distinguished name components. Each element is a table with four fields:

field	description
.sn	short name identifier, if available
.ln	long name identifier, if available
.id	OID identifier
.blob	raw string value of the component

`name:each()`

Returns a key-value iterator over the distinguished name components. The key is either the short, long, or OID identifier, with preference for the former.

`name:__pairs()`

Equal to `name:each`

3.1.5 openssl.x509.altname

Binds the X.509 alternative names (a.k.a “general names”) OpenSSL ASN.1 object, used for representing certificate subject and issuer alternative names.

`altname.new()`

Returns an empty altname object.

`altname.interpose(name, function)`

Add or interpose an altname class method. Returns the previous method, if any.

`altname:add(type, value)`

Add an alternative name. *type* must specify one of the five basic types identified by “RFC822Name”, “RFC822”, “email”, “UniformResourceIdentifier”, “URI”, “DNSName”, “DNS”, “IPAddress”, “IP”, or “DirName”.

For all types except “DirName”, *value* is a string acceptable to OpenSSL’s sanity checks. For an IP address, *value* must be parseable by the system’s `inet_pton` routine, as IP addresses are stored as raw 4- or 16-byte octets. “DirName” takes an `openssl.x509.name` object.

`name:__pairs()`

Returns a key-value iterator over the alternative names. The key is one of “email”, “URI”, “DNS”, “IP”, or “DirName”. The value is the string representation of the name.

3.1.6 `openssl.x509.extension`

Binds the X.509 extension OpenSSL object.

`extension.new(name, value [, data])`

Returns a new X.509 extension. If *value* is the string “DER” or “critical,DER”, then *data* is an ASN.1-encoded octet string. Otherwise, *name* and *value* are plain text strings in **OpenSSL’s arbitrary extension format**; and if specified, *data* is either an OpenSSL configuration string defining any referenced identifiers in *value*, or a table with members:

field	type:default	description
.db	string: <i>nil</i>	OpenSSL configuration string
.issuer	<code>openssl.x509:nil</code>	issuer certificate
.subject	<code>openssl.x509:nil</code>	subject certificate
.request	<code>openssl.x509.csr:nil</code>	certificate signing request
.crl	<code>openssl.x509.crl:nil</code>	certificate revocation list
.flags	integer:0	a bitwise combination of flags

`extension.interpose(name, function)`

Add or interpose an extension class method. Returns the previous method, if any.

`extension:getID()`

Returns the ASN.1 OID as a plain text string.

`extension:getName()`

Returns a more human-readable name as a plain text string in the following order of preference: OpenSSL’s short name, OpenSSL’s long name, ASN.1 OID.

`extension:getShortName()`

Returns OpenSSL’s short name as a plain text string if available.

`extension:getLongName()`

Returns OpenSSL’s long name as a plain text string if available.

`extension:getData()`

Returns the extension value as an ASN.1-encoded octet string.

`extension:getCritical()`

Returns the extension critical flag as a boolean.

3.1.7 `openssl.x509`

Binds the X.509 certificate OpenSSL ASN.1 object.

`x509.new([string[, format]])`

Returns a new x509 object, optionally initialized to the PEM- or DER-encoded certificate specified by *string*. *format* is as described in `openssl.pkey.new`—“PEM”, “DER”, or “*” (default).

`x509.interpose(name, function)`

Add or interpose an x509 class method. Returns the previous method, if any.

`x509:getVersion()`

Returns the X.509 version of the certificate.

`x509:setVersion(number)`

Sets the X.509 version of the certificate.

`x509:getSerial()`

Returns the serial of the certificate as an `openssl.bignum`.

`x509:setSerial(number)`

Sets the serial of the certificate. *number* is a Lua or `openssl.bignum` number.

`x509:digest([type[, format]])`

Returns the cryptographic one-way message digest of the certificate. *type* is the OpenSSL string identifier of the hash type—e.g. “md5”, “sha1” (default), “sha256”, etc. *format* specifies the representation of the digest—“s” for an octet string, “x” for a hexadecimal string (default), and “n” for an `openssl.bignum` number.

`x509:getLifetime()`

Returns the certificate validity “Not Before” and “Not After” dates as two Unix timestamp numbers.

`x509:setLifetime([notbefore][, notafter])`

Sets the certificate validity dates. *notbefore* and *notafter* should be UNIX timestamps. A nil value leaves the particular date unchanged.

`x509:getIssuer()`

Returns the issuer distinguished name as an `x509.name` object.

`x509:setIssuer(name)`

Sets the issuer distinguished name.

`x509:getSubject()`

Returns the subject distinguished name as an `x509.name` object.

`x509:setSubject(name)`

Sets the subject distinguished name.

`x509:getIssuerAlt()`

Returns the issuer alternative names as an `x509.altname` object.

`x509:setIssuer(altname)`

Sets the issuer alternative names.

`x509:getSubjectAlt()`

Returns the subject alternative names as an `x509.name` object.

`x509:setSubjectAlt(name)`

Sets the subject alternative names.

`x509:getIssuerAltCritical()`

Returns the issuer alternative names critical flag as a boolean.

`x509:setIssuerAltCritical(boolean)`

Sets the issuer alternative names critical flag.

`x509:getSubjectAltCritical()`

Returns the subject alternative names critical flag as a boolean.

`x509:setSubjectAltCritical(boolean)`

Sets the subject alternative names critical flag.

`x509:getBasicConstraints([which[], which ... ])`

Returns the X.509 ‘basic constraint’ flags. If specified, *which* should be one of “CA” or “pathLen”, which returns the specified constraint—respectively, a boolean and a number. If no parameters are specified, returns a table with fields “CA” and “pathLen”.

`x509:setBasicConstraints{ ... }`

Sets the basic constraint flag according to the defined field values for “CA” (boolean) and “pathLen” (number).

`x509:getBasicConstraintsCritical()`

Returns the basic constraints critical flag as a boolean.

`x509:setBasicConstraintsCritical(boolean)`

Sets the basic constraints critical flag.

`x509:addExtension(ext)`

Adds a copy of the `x509.extension` object to the certificate.

`x509:getExtension(key)`

Returns a copy of the `x509.extension` object identified by *key* where *key* is the plain text string of the OID, long name, or short name; or the integer index (1-based) of the extension. Returns nothing if no such extension was found by that name or at that index.

`x509:getExtensionCount()`

Returns the integer count of the number of extensions.

`x509:getOCSP()`

Returns the OCSP urls for the certificate.

`x509:isIssuedBy(issuer)`

Returns a boolean according to whether the specified issuer—an `openssl.x509.name` object—signed the instance certificate.

`x509:getPublicKey()`

Returns the public key component as an `openssl.pkey` object.

`x509:setPublicKey(key)`

Sets the public key component referenced by the `openssl.pkey` object *key*.

`x509:getPublicKeyDigest([type])`

Returns the digest of the public key as a binary string. *type* is an optional string describing the digest type, and defaults to “sha1”.

`x509:getSignatureName()`

Returns the type of signature used to sign the certificate as a string. e.g. “RSA-SHA1”

`x509:sign(key [, type])`

Signs and updates the instance certificate using the `openssl.pkey` *key*. *type* is an optional string describing the digest type. See `pkey:sign`, regarding which types of digests are valid. If *type* is omitted than a default type is used—“sha1” for RSA keys, “dss1” for DSA keys, and “ecdsa-with-SHA1” for EC keys.

`x509:verify{ ... }`

Verifies the certificate against to the specified parameters.

field	type	description
<code>.store</code>	<code>openssl.x509.store</code>	The certificate store to verify against, any custom settings from the store will be used.
<code>.chain</code>	<code>openssl.x509.chain</code>	A collection of additional certificates to consider
<code>.params</code>	<code>openssl.x509.verify_param</code>	The verification parameters to use; overrides any parameters in <i>.store</i>

Returns two values. The first is a boolean value for whether the specified certificate *crt* was verified. If true, the second value is a `openssl.x509.chain` object validation chain. If false, the second value is a string describing why verification failed.

`x509:text()`

Returns a human-readable textual representation of the X.509 certificate.

`x509:__toString`

Returns the PEM encoded representation of the instance certificate.

3.1.8 openssl.x509.csr

Binds the X.509 certificate signing request OpenSSL ASN.1 object.

`csr.new([x509|string[, format]])`

Returns a new request object, optionally initialized to the specified `openssl.x509` certificate *x509* or the PEM- or DER-encoded certificate signing request *string*. *format* is as described in `openssl.pkey.new`—“PEM”, “DER”, or “*” (default).

`csr.interpose(name, function)`

Add or interpose a request class method. Returns the previous method, if any.

`csr.getVersion()`

Returns the X.509 version of the request.

`csr:setVersion(number)`

Sets the X.509 version of the request.

`csr:getSubject()`

Returns the subject distinguished name as an `x509.name` object.

`csr:setSubject(name)`

Sets the subject distinguished name. *name* should be an `x509.name` object.

`csr:getSubjectAlt()`

Returns the subject alternative name as an `x509.altname` object.

`csr:setSubjectAlt(name)`

Sets the subject alternative names. *name* should be an `x509.altname` object.

`csr:getPublicKey()`

Returns the public key component as an `openssl.pkey` object.

`csr:setPublicKey(key)`

Sets the public key component referenced by the `openssl.pkey` object *key*.

`csr:sign(key)`

Signs the instance request using the `openssl.pkey` *key*.

`csr:__tostring`

Returns the PEM encoded representation of the instance request.

3.1.9 `openssl.x509.crl`

Binds the X.509 certificate revocation list OpenSSL ASN.1 object.

`crl.new([string[, format]])`

Returns a new CRL object, optionally initialized to the specified PEM- or DER-encoded CRL *string*. *format* is as described in `openssl.pkey.new`—“PEM”, “DER”, or “*” (default).

`crl.interpose(name, function)`

Add or interpose a request class method. Returns the previous method, if any.

`crl.getVersion()`

Returns the CRL version.

`crl:setVersion(number)`

Sets the CRL version.

`crl:getLastUpdate()`

Returns the Last Update time of the CRL as a Unix timestamp, or *nil* if not set.

`crl:setLastUpdate(time)`

Sets the Last Update time of the CRL. *time* should be a Unix timestamp.

`crl:getNextUpdate()`

Returns the Next Update time of the CRL as a Unix timestamp, or *nil* if not set.

`crl:setNextUpdate(time)`

Sets the Next Update time of the CRL. *time* should be a Unix timestamp.

`crl:getIssuer()`

Returns the issuer distinguished name as an `x509.name` object.

`crl:setIssuer(name)`

Sets the issuer distinguished name. *name* should be an `x509.name` object.

`crl:add(serial [, time])`

Add the certificate identified by *serial* to the revocation list. *serial* should be a `openssl.bignum` object, as returned by `x509:getSerial`. *time* is the revocation date as a Unix timestamp. If unspecified *time* defaults to the current time.

`crl:addExtension(ext)`

Adds a copy of the `x509.extension` object to the revocation list.

`crl:getExtension(key)`

Returns a copy of the `x509.extension` object identified by *key* where *key* is the plain text string of the OID, long name, or short name; or the integer index (1-based) of the extension. Returns nothing if no such extension was found by that name or at that index.

`crl:getExtensionCount()`

Returns the integer count of the number of extensions.

`crl:sign(key)`

Signs the instance CRL using the `openssl.pkey` *key*.

`crl:verify(publickey)`

Verifies the instance CRL using a public key.

`crl:text()`

Returns a human-readable textual representation of the instance CRL.

`crl:__toString`

Returns the PEM encoded representation of the instance CRL.

3.1.10 `openssl.x509.chain`

Binds the “STACK_OF(X509)” OpenSSL object, principally used in the OpenSSL library for representing a validation chain.

`chain.new()`

Returns a new chain object.

`chain.interpose(name, function)`

Add or interpose a chain class method. Returns the previous method, if any.

`chain.add(crt)`

Append the X.509 certificate *crt*.

`chain.__ipairs()`

Returns an iterator over the stored certificates.

3.1.11 `openssl.x509.store`

Binds the X.509 certificate “X509_STORE” OpenSSL object, principally used for loading and storing trusted certificates, paths to trusted certificates, and verification policy.

`store.new()`

Returns a new store object.

`store.interpose(name, function)`

Add or interpose a store class method. Returns the previous method, if any.

`store.add(crt|filepath|dirpath)`

Add the X.509 certificate *crt* to the store, load the certificates from the file *filepath*, or set the OpenSSL ‘hashdir’ certificate path *dirpath*.

`store.verify(crt[, chain])`

Returns two values. The first is a boolean value for whether the specified certificate *crt* was verified. If true, the second value is a `openssl.x509.chain` object validation chain. If false, the second value is a string describing why verification failed. The optional parameter *chain* is an `openssl.x509.chain` object of untrusted certificates linking the certificate *crt* to one of the trusted certificates in the instance store.

3.1.12 `openssl.x509.verify_param`

Binds the “X509_VERIFY_PARAM” OpenSSL object, principally used for setting parameters to be used during certificate verification operations.

`verify_param.new()`

Returns a new `verify_param` object.

`verify_param.interpose(name, function)`

Add or interpose a `verify_param` class method. Returns the previous method, if any.

`verify_param:inherit(src)`

Inherit flags from *src*. *src* can be either another `verify_param` object to inherit from, or a string referring to one of the OpenSSL predefined parameters:

name	description
default	X509 default parameters
smime_sign	S/MIME sign parameters
pkcs7	Identical to <i>smime_sign</i>
ssl_client	SSL/TLS client parameters
ssl_server	SSL/TLS server parameters

`verify_param:setPurpose(id_or_name)`

Sets the verification purpose of the *verify_param*. Valid argument can be either an integer which corresponds to OpenSSL's internal purpose ID, or string indicating predefined purposes:

name	description
sslclient	SSL/TLS client
sslserver	SSL/TLS server
nsslsrserver	Netscape SSL server
smimeencrypt	S/MIME encryption
any	Any Purpose
ocsp-helper	OCSP helper
timestampsign	Time Stamp signing

`verify_param:setTime([timestamp])`

Sets the verification time in *verify_param* to the provided Unix timestamp. By default the current system time is used.

`verify_param:setDepth(depth)`

Sets the maximum verification depth to *depth*. That is the maximum number of untrusted CA certificates that can appear in a chain.²

`verify_param:getDepth()`

Returns the current maximum verification depth.

²OpenSSL's behaviour in regards to depth changed between OpenSSL 1.0.1 and OpenSSL 1.0.2; similarly for LibreSSL

`verify_param:setAuthLevel(auth_level)`

Sets the authentication security level to *auth_level*. The authentication security level determines the acceptable signature and public key strength when verifying certificate chains. For a certificate chain to validate, the public keys of all the certificates must meet the specified security level. The signature algorithm security level is not enforced for the chain's trust anchor certificate, which is either directly trusted or validated by means other than its signature. See [*SSL_CTX_set_security_level\(3\)*](#) for the definitions of the available levels. The default security level is -1, or "not set". At security level 0 or lower all algorithms are acceptable. Security level 1 requires at least 80-bit-equivalent security and is broadly interoperable, though it will, for example, reject MD5 signatures or RSA keys shorter than 1024 bits.

Only supported since OpenSSL 1.1.0.

`verify_param:getAuthLevel()`

Returns the current authentication security level.

Only supported since OpenSSL 1.1.0.

`verify_param:setHost(name)`

Sets the expected DNS hostname to the string *name*, overriding any previously specified host name or names. If *name* is *nil* then name checks will not be performed on the peer certificate.

Only supported since OpenSSL 1.1.0.

`verify_param:addHost(name)`

Adds *name* as an additional reference identifier that can match the peer's certificate. Any previous names set via `verify_param:setHost` or `verify_param:addHost` are retained. When multiple names are configured, the peer is considered verified when any name matches.

Only supported since OpenSSL 1.1.0.

`verify_param:setEmail(email)`

Sets the expected RFC822 email address to the string *email*, overriding any previously specified email address.

Only supported since OpenSSL 1.1.0.

`verify_param:setIP(address)`

Sets the expected IP address to *address*. Can be dotted decimal quad for IPv4 and colon-separated hexadecimal for IPv6. The condensed "::" notation is supported for IPv6 addresses.

Only supported since OpenSSL 1.1.0.

3.1.13 openssl.pkcs12

Binds the PKCS #12 container OpenSSL object.

`pkcs12.new{ ... }`

Returns a new PKCS12 object initialized according to the named parameters—*password*, *key*, *certs*.

FIXME.

`pkcs12.interpose(name, function)`

Add or interpose a store class method. Returns the previous method, if any.

`pkcs12:__toString()`

Returns a PKCS #12 binary encoded string.

`pkcs12.parse(bag[, passphrase])`

Parses a PKCS#12 bag, presented as a binary string *bag*. The second parameter *passphrase* is the passphrase required to decrypt the PKCS#12 bag. The function returns three items; namely the key, certificate and the CA chain, as their respective objects. If an item is absent, it will be substituted with nil.

3.1.14 openssl.ssl.context

Binds the “SSL_CTX” OpenSSL object, used as a configuration prototype for SSL connection instances. See `socket.starttls`.

`context[]`

A table mapping OpenSSL named constants. The available constants are documented with the relevant method.

`context.new([protocol][, server])`

Returns a new context object. *protocol* is an optional string identifier selecting the OpenSSL constructor, defaulting to “TLS”. If *server* is true, then SSL connections instantiated using this context will be placed into server mode, otherwise they behave as clients.

<i>protocol</i> identifiers	
name	description
TLS	Supports TLS 1.0 <i>and above</i> . Internally uses <code>SSLv23_method</code> and disables <code>SSLv2</code> and <code>SSLv3</code> using <code>SSL_OP_NO_SSLv2</code> and <code>SSL_OP_NO_SSLv3</code> .
SSL	Supports SSL 3.0 <i>and above</i> . Internally uses <code>SSLv23_method</code> and disables <code>SSLv2</code> using <code>SSL_OP_NO_SSLv2</code> .
SSLv23	A catchall for all versions of SSL/TLS supported by OpenSSL. Individual versions can be disabled using <code>context:setOptions</code> . Internally uses <code>SSLv23_method</code> .
TLSv1.2	Supports <i>only</i> TLS 1.2. Internally uses <code>TLSv1.2_method</code> .
TLSv1.1	Supports <i>only</i> TLS 1.1. Internally uses <code>TLSv1.1_method</code> .

TLSv1	Supports <i>only</i> TLS 1.0. Internally uses <code>TLSv1_method</code> .
SSLv3	Supports <i>only</i> SSL 3.0. Internally uses <code>SSLv3_method</code> .
SSLv2	Supports <i>only</i> SSL 2.0. Internally uses <code>SSLv2_method</code> .
DTLS	Supports DTLS 1.0 <i>and above</i> . Internally uses <code>DTLS_method</code> .
DTLSv1	Supports <i>only</i> DTLS 1.0. Internally uses <code>DTLSv1_method</code> .
DTLSv1.2	Supports <i>only</i> DTLS 1.2. Internally uses <code>DTLSv1_2_method</code> .

`context.interpose(name, function)`

Add or interpose a context class method. Returns the previous method, if any.

`context.setOptions(flags)`

Adds the option flags to the context instance. *flags* is a bit-wise set of option flags to be OR'd with the current set. The resultant option flags of the context instance will be the union of the old and new flags.³

name	description
<code>OP_MICROSOFT_SESS_ID_BUG</code>	When talking SSLv2, if session-id reuse is performed, the session-id passed back in the server-finished message is different from the one decided upon.
<code>OP_NETSCAPE_CHALLENGE_BUG</code>	Workaround for Netscape-Commerce/1.12 servers.
<code>OP_LEGACY_SERVER_CONNECT</code>	...
<code>OP_NETSCAPE_REUSE_CIPHER_CHANGE_BUG</code>	As of OpenSSL 0.9.8q and 1.0.0c, this option has no effect.
<code>OP_SSLREF2_REUSE_CERT_TYPE_BUG</code>	...
<code>OP_TLS_EXT_PADDING</code>	...
<code>OP_MICROSOFT_BIG_SSLV3_BUFFER</code>	...
<code>OP_SAFARI_ECDHE_ECDSA_BUG</code>	...
<code>OP_MSIE_SSLV2_RSA_PADDING</code>	...
<code>OP_SSLEAY_080_CLIENT_DH_BUG</code>	...
<code>OP_TLS_D5_BUG</code>	...
<code>OP_TLS_BLOCK_PADDING_BUG</code>	...
<code>OP_ALLOW_NO_DHE_KEX</code>	Allow a non-(ec)dhe based <code>kex_mode</code> .

³This idiosyncratic union behavior is how the OpenSSL routine works.

OP_DONT_INSERT_EMPTY_FRAGMENTS	Disables a countermeasure against a SSL 3.0/TLS 1.0 protocol vulnerability affecting CBC ciphers, which cannot be handled by some broken SSL implementations. This option has no effect for connections using other ciphers.
OP_NO_QUERY_MTU	...
OP_COOKIE_EXCHANGE	...
OP_NO_TICKET	Disable RFC4507bis ticket stateless session resumption.
OP_CISCO_ANYCONNECT	...
OP_NO_SESSION_RESUMPTION_ON_RENEGOTIATION	When performing renegotiation as a server, always start a new session (i.e., session resumption requests are only accepted in the initial handshake). This option is not needed for clients.
OP_NO_COMPRESSION	...
OP_ALLOW_UNSAFE_LEGACY_RENEGOTIATION	...
OP_SINGLE_ECDH_USE	Always create a new key when using temporary/ephemeral ECDH parameters.
OP_NO_ENCRYPT_THEN_MAC	...
OP_SINGLE_DH_USE	Always create a new key when using temporary/ephemeral DH parameters.
OP_EPHEMERAL_RSA	Always use ephemeral (temporary) RSA key when doing RSA operations.
OP_PRIORITIZE_CHACHA	Prioritize ChaCha20Poly1305 on servers when client does.
OP_ENABLE_MIDDLEBOX_COMPAT	TLSv1.3 Compatibility mode.
OP_NO_ANTI_REPLAY	TLSv1.3 anti-replay protection for early data.
OP_CIPHER_SERVER_PREFERENCE	When choosing a cipher, use the server's preferences instead of the client preferences.
OP_TLS_ROLLBACK_BUG	Disable version rollback attack detection.
OP_NO_SSLv2	Do not use the SSLv2 protocol.
OP_NO_SSLv3	Do not use the SSLv3 protocol.
OP_NO_TLSv1/OP_NO_DTLSv1	Do not use the (D)TLSv1.0 protocol.
OP_NO_TLSv1_2/OP_NO_DTLSv1_2	Do not use the (D)TLSv1.2 protocol.
OP_NO_TLSv1_1	Do not use the TLSv1.1 protocol.
OP_NETSCAPE_CA_DN_BUG	...
OP_NO_TLSv1_3	...
OP_NETSCAPE_DEMO_CIPHER_CHANGE_BUG	...
OP_NO_RENEGOTIATION	...
OP_CRYPTOPRO_TLSEXT_BUG	...

OP_PKCS1_CHECK_1	...
OP_PKCS1_CHECK_2	...
OP_NO_SSL_MASK	...
OP_ALL	All of the bug workarounds.

`context:getOption()`

Returns the option flags of the context instance as an integer.

`context:clearOptions()`

Clears the option flags of the context instance.

`context:setReadAhead(yes)`

Sets if read ahead is enabled for the context, *yes* should be a boolean.

`context:getReadAhead()`

Returns if read ahead is enabled for the context instance as a boolean.

`context:setStore(store)`

Associate the `openssl.x509.store` object *store* with *context*. Replaces any existing store.

`context:getStore()`

Returns the `openssl.x509.store` object associated with *context*.

`context:setParam(params)`

Causes *context* to inherit the parameters from the `openssl.x509.verify_param` object *params*. Only parameters set in *params* will take effect (others will stay unchanged).

`context:getParam()`

Returns an `openssl.x509.verify_param` object containing a copy of *context*'s parameters.

`context:setVerify([mode][, depth])`

Sets the verification mode flags and maximum validation chain depth.

name	description
VERIFY_NONE	disable client peer certificate verification
VERIFY_PEER	enable client peer certificate verification
VERIFY_FAIL_IF_NO_PEER_CERT	require a peer certificate
VERIFY_CLIENT_ONCE	do not request peer certificates after initial handshake

See the [NOTES section](#) in the OpenSSL documentation for `SSL_CTX_set_verify_mode`.

`context:verify()`

Returns two values: the bitwise verification mode flags, and the maximum validation depth.

`context:setCertificate(cert)`

Sets the X.509 certificate `openssl.x509` object *cert* to send during SSL connection instance handshakes.

`context:getCertificate()`

Returns the X.509 certificate `openssl.x509` object to be sent during SSL connection instance handshakes.

Only supported since OpenSSL 1.0.2.

`context:setCertificateChain(chain)`

Sets the X.509 certificate chain `openssl.x509.chain` object *chain* to send during SSL connection instance handshakes.

Only supported since OpenSSL 1.0.2.

`context:getCertificateChain()`

Returns the X.509 certificate chain `openssl.x509.chain` object to be sent during SSL connection instance handshakes.

Only supported since OpenSSL 1.0.2.

`context:setPrivateKey(key)`

Sets the private key `openssl.pkey` object *key* for use during SSL connection instance handshakes.

`context:setCipherList(string [, ...])`

Sets the allowed public key and private key algorithm(s). The string format is documented in the [OpenSSL ciphers\(1\) utility documentation](#).

`context:setCipherSuites(string [, ...])`

Sets the supported TLS 1.3 cipher suites. The string format is a list of colon separated curve names similar to `ctx:setCipherList(...)`.

Only supported since OpenSSL 1.1.1.

`context:setGroups(string [, ...])`

Sets the supported groups. The string format is a list of colon separated group names similar to `ctx:setCipherList(...)`. A list of supported EC groups can be found by running `openssl ecparam -list_curves`.

Only supported since OpenSSL 1.0.2.

`context:setEphemeralKey(key)`

Sets `openssl.pkey` object *key* as the ephemeral key during key exchanges which use that particular key type. Typically *key* will be either a Diffie-Hellman or Elliptic Curve key.

In older version of OpenSSL, in order to configure an SSL server to support an ephemeral key exchange cipher suite (i.e. DHE- and ECDHE-*), the application must explicitly set the ephemeral keys. Simply enabling the cipher suite is not sufficient. The application can statically generate Diffie-Hellman public key parameters, and many servers ship with such a key compiled into the software. Elliptic curve keys are necessarily static, and instantiated by curve name⁴.*

In addition, to attain Perfect Forward Secrecy the options `OP_SINGLE_DH_USE` and `OP_SINGLE_ECDH_USE` must be set so that OpenSSL discards and regenerates the secret keying parameters for each key exchange.

`context:setAlpnProtos(table)`

Sets the advertised ALPN protocols. *table* is an array of protocol string identifiers.

Only supported since OpenSSL 1.0.2.

`context:setAlpnSelect(cb)`

Sets the callback used to select an ALPN protocol. *cb* should be a function that takes two arguments: an `openssl.ssl` object and a table containing a sequence of ALPN protocol strings; it should return the ALPN protocol string it selected or *nil* to select none of them.

Only supported since OpenSSL 1.0.2.

`context:setHostNameCallback(cb)`

Sets the callback used to process the SNI in a ClientHello. *cb* should be a function that one argument: a `openssl.ssl` object; it should return *true* to indicate success, *false* if no acknowledgement should be send to the client, or *nil* and an integer to send an error to the peer.

Only supported since OpenSSL 1.0.0.

⁴OpenSSL 1.0.2 only supports a single curve, [according to Wikipedia](#) the most widely supported curve is prime256v1, so to enable ECDHE-*, applications can simply do `ctx:setEphemeralKey(pkey.new{ type = 'EC', curve = 'prime256v1' })`. For OpenSSL versions $\geq$ 1.0.2, see `context:setGroups` instead. To achieve Perfect Forward Secrecy for ECDHE-*, applications must also do `ctx:setOptions(context.OP_SINGLE_ECDH_USE)`. The `ctx` object must then be used to configure each SSL session, such as by passing it to `cqueues.socket:starttls()`.

`context:setTLSextStatusType(type)`

Sets the default TLS extension status for SSL objects derived from this context. See `ssl:setTLSextStatusType`
Only supported since OpenSSL 1.1.0.

`context:getTLSextStatusType()`

Gets the default TLS extension status for SSL objects derived from this context as a string. See `ssl:getTLSextStatusType`

Only supported since OpenSSL 1.1.0.

`context:getTicketKeysLength()`

Returns the expected length of ticket keys data. See `context:setTicketKeys`

Only supported since OpenSSL 1.0.0.

`context:setTicketKeys(keys)`

Sets the current random data used to generate tickets. *keys* should be a string of the length returned by `context:getTicketKeysLength`.

Only supported since OpenSSL 1.0.0.

`context:getTicketKeys()`

Returns the current random data used to generate tickets. See `context:setTicketKeys`

Only supported since OpenSSL 1.0.0.

`context:useServerInfo(version, serverinfo)`

If *version* is 1 then the extensions in the array must consist of a 2-byte Extension Type, a 2-byte length, and then length bytes of extension data. The type value has the same meaning as for `context:addCustomExtension`.

If *version* is 2 then the extensions in the array must consist of a 4-byte context, a 2-byte Extension Type, a 2-byte length, and then length bytes of extension data. The context and type values have the same meaning as for `context:addCustomExtension`. If *serverinfo* is being loaded for extensions to be added to a Certificate message, then the extension will only be added for the first certificate in the message (which is always the end-entity certificate).

Only supported since OpenSSL 1.0.2, ServerInfo version 2 is only supported since OpenSSL 1.1.1

`context:useServerInfoFile(file)`

Loads one or more *serverinfo* extensions from *file* into *context*. The extensions must be in PEM format. Each extension must be in a format as described above for `context:useServerInfo`. Each PEM extension name must begin with the phrase “BEGIN SERVERINFOV2 FOR ” for version 2 data or “BEGIN SERVERINFO FOR ” for version 1 data.

Only supported since OpenSSL 1.0.2

`context:addCustomExtension(ext_type, ext_context, add_cb, parse_cb)`

Adds a custom extension with the TLS extension type (see RFC 5246) *ext_type* that may be present in the context(s) specified by *ext_context*, which should be a bitmask of the flags:

name	description
EXT_TLS_ONLY	The extension is only allowed in TLS
EXT_DTLS_ONLY	The extension is only allowed in DTLS
EXT_TLS_IMPLEMENTATION_ONLY	The extension is allowed in DTLS, but there is only a TLS implementation available (so it is ignored in DTLS).
EXT_SSL3_ALLOWED	Extensions are not typically defined for SSLv3. Setting this value will allow the extension in SSLv3. Applications will not typically need to use this.
EXT_TLS1_2_AND_BELOW_ONLY	The extension is only defined for TLSv1.2/DTLSv1.2 and below. Servers will ignore this extension if it is present in the ClientHello and TLSv1.3 is negotiated.
EXT_TLS1_3_ONLY	The extension is only defined for TLS1.3 and above. Servers will ignore this extension if it is present in the ClientHello and TLSv1.2 or below is negotiated.
EXT_IGNORE_ON_RESUMPTION	The extension will be ignored during parsing if a previous session is being successfully resumed.
EXT_CLIENT_HELLO	The extension may be present in the ClientHello message.
EXT_TLS1_2_SERVER_HELLO	The extension may be present in a TLSv1.2 or below compatible ServerHello message.
EXT_TLS1_3_SERVER_HELLO	The extension may be present in a TLSv1.3 compatible ServerHello message.
EXT_TLS1_3_ENCRYPTED_EXTENSIONS	The extension may be present in an EncryptedExtensions message.
EXT_TLS1_3_HELLO_RETRY_REQUEST	The extension may be present in a HelloRetryRequest message.
EXT_TLS1_3_CERTIFICATE	The extension may be present in a TLSv1.3 compatible Certificate message.
EXT_TLS1_3_NEW_SESSION_TICKET	The extension may be present in a TLSv1.3 compatible NewSessionTicket message.
EXT_TLS1_3_CERTIFICATE_REQUEST	The extension may be present in a TLSv1.3 compatible CertificateRequest message.

add_cb should be a function with signature `add_cb(ssl, ext_type, ext_context, x509, chainidx)`; it will be called from the relevant context to allow you to insert extension data. It receives the *ssl* object of the connection, the *ext_type* you registered the callback for, the current *context* and, for only some contexts, the current `openssl.x509` certificate and chain index (as an integer). You should return the extension data as a string, *false* if you don't want to add your extension, or *nil* and an optional integer specifying the TLS error code to raise an error.

parse_cb should be a function with signature `parse_cb(ssl, ext_type, ext_context, data, x509, chainidx)`; it will be called from the relevant context to allow you to parse extension data. It receives the *ssl* object of the connection, the *ext_type* you registered the callback for, the current *context*, the extension *data* as a string, and for only some contexts, the current `openssl.x509` certificate and chain index (as an integer). You should return *true* on success, or *nil* and an optional integer specifying the TLS error code to raise an error.

Only supported since OpenSSL 1.1.1.

3.1.15 `openssl.ssl`

Binds the “SSL” OpenSSL object, which represents an SSL connection instance. See `cqueues.socket:checktls`.

`ssl[]`

A table mapping OpenSSL named constants. Includes all constants provided by `openssl.ssl.context`. Additional constants are documented with the relevant method.

`ssl.interpose(name, function)`

Add or interpose an ssl class method. Returns the previous method, if any.

`ssl:setContext(context)`

Replaces the `openssl.ssl.context` used by *ssl* with *context*.

`ssl:getContext()`

Returns the `openssl.ssl.context` used by *ssl*.

`ssl:setOptions(flags)`

Adds the option flags of the SSL connection instance. See `openssl.ssl.context:setOptions`.

`ssl:getOptions()`

Returns the option flags of the SSL connection instance. See `openssl.ssl.context:getOptions`.

`ssl:clearOptions()`

Clears the option flags of the SSL connection instance. See `openssl.ssl.context:clearOptions`.

`ssl:setReadAhead(yes)`

Sets if read ahead is enabled for the SSL connection instance, *yes* should be a boolean.

`ssl:getReadAhead()`

Returns if read ahead is enabled for the SSL connection instance as a boolean.

`ssl:setStore(store)`

Associate the `openssl.x509.store` object *store* with *ssl* for both verification and chain building. Replaces any existing stores.

Only supported since OpenSSL 1.0.2.

`ssl:setChainStore(store)`

Associate the `openssl.x509.store` object *store* with *ssl* for chain building. Replaces any existing store.

Only supported since OpenSSL 1.0.2.

`ssl:setVerifyStore(store)`

Associate the `openssl.x509.store` object *store* with *ssl* for verification. Replaces any existing store.

Only supported since OpenSSL 1.0.2.

`ssl:setVerify([mode][, depth])`

Sets the verification mode flags and maximum validation chain depth. See `openssl.ssl.context:setVerify`.

`ssl:getVerify()`

Returns two values: the bitwise verification mode flags, and the maximum validation depth. See `openssl.ssl.context:getVerify`.

`ssl:getVerifyResult()`

Returns two values: the integer verification result code and the string representation of that code.

`ssl:setCertificate(crt)`

Sets the X.509 certificate `openssl.x509` object *crt* to send during SSL connection instance handshakes. See `openssl.ssl.context:setCertificate`.

`ssl:setCertificateChain(chain)`

Sets the X.509 certificate chain `openssl.x509.chain` object *chain* to send during SSL connection instance handshakes. See `openssl.ssl.context:setCertificateChain`.

Only supported since OpenSSL 1.0.2.

`context:getCertificateChain()`

Returns the X.509 certificate chain `openssl.x509.chain` object to be sent during SSL connection instance handshakes. See `openssl.ssl.context:getCertificateChain`.

Only supported since OpenSSL 1.0.2.

`ssl:setPrivateKey(key)`

Sets the private key `openssl.pkey` object *key* for use during SSL connection instance handshakes. See `openssl.ssl.context:setPrivateKey`.

`ssl:getPeerCertificate()`

Returns the X.509 peer certificate as an `openssl.x509` object. If no peer certificate is available, returns *nil*.

`ssl:getPeerChain()`

Similar to `:getPeerCertificate`, but returns the entire chain sent by the peer as an `openssl.x509.chain` object.

`ssl:getCipherInfo()`

Returns a table of information on the current cipher.

field	description
<code>.name</code>	cipher name returned by <code>SSL_CIPHER_get_name</code>
<code>.bits</code>	number of secret bits returned by <code>SSL_CIPHER_get_bits</code>
<code>.version</code>	SSL/TLS version string returned by <code>SSL_CIPHER_get_version</code>
<code>.description</code>	key:value cipher description returned by <code>SSL_CIPHER_description</code>

`ssl:setHostName(host)`

Sets the Server Name Indication (SNI) host name. Using the SNI TLS extension, clients tell the server which domain they're contacting so the server can select the proper certificate and key. This permits SSL virtual hosting. This routine is only relevant for clients.

`ssl:getHostName()`

Returns the Server Name Indication (SNI) host name sent by the client. If no host name was sent, returns *nil*. This routine is only relevant for servers.

`ssl: getVersion([format])`

Returns the SSL/TLS version of the negotiated SSL connection. By default returns a 16-bit integer where the top 8 bits are the major version number and the bottom 8 bits the minor version number. For example, SSL 3.0 is 0x0300 and TLS 1.1 is 0x0302. SSL 2.0 is 0x0002.

If *format* is “.” returns a floating point number. 0x0300 becomes 3.0, and 0x0302 becomes 3.2. If the minor version is ≥ 10 an error is thrown.⁵

The following OpenSSL named constants can be used.

name	description
SSL2_VERSION	16-bit SSLv2 identifier (0x0002).
SSL3_VERSION	16-bit SSLv3 identifier (0x0300).
TLS1_VERSION	16-bit TLSv1.0 identifier (0x0301).
TLS1_1_VERSION	16-bit TLSv1.1 identifier (0x0302).
TLS1_2_VERSION	16-bit TLSv1.2 identifier (0x0303).

`ssl: getClientVersion([format])`

Returns the SSL/TLS version supported by the client, which should be greater than or equal to the negotiated version. See `ssl: getVersion`.

`ssl: setCipherList(string [, ...])`

Sets the allowed public key and private key algorithm(s). See `openssl.ssl.context:setCipherList`.

`ssl: setCipherSuites(string [, ...])`

Sets the supported TLS 1.3 cipher suites for this SSL connection instance. See `openssl.ssl.context:setCipherSuites`.

Only supported since OpenSSL 1.1.1.

`ssl: setGroups(string [, ...])`

Sets the supported groups for this SSL connection instance. See `openssl.ssl.context:setGroups`.

Only supported since OpenSSL 1.0.2.

`ssl: getAlpnSelected()`

Returns the negotiated ALPN protocol as a string.

Only supported since OpenSSL 1.0.2.

`ssl: setAlpnProtos(table)`

Sets the advertised ALPN protocols. *table* is an array of protocol string identifiers.

Only supported since OpenSSL 1.0.2.

⁵This condition shouldn't be possible.

`ssl:setTLSextStatusType(type)`

Sets the TLS extension status.

Only the *type* “ocsp” is currently supported, this is used by a client to request that a server sends a stapled OCSF response as part of the TLS handshake.

See also: `context:setTLSextStatusType()`

`ssl:getTLSextStatusType()`

Gets the TLS extension status. As set by `ssl:setTLSextStatusType` or `context:setTLSextStatusType`.

Only the type “ocsp” is currently known.

Only supported since OpenSSL 1.1.0.

`ssl:setTLSextStatusOCSPResp(or)`

Sets an `openssl.ocsp.response`. Used by a server to staple an OCSF response into a TLS handshake.

`ssl:getTLSextStatusOCSPResp()`

Returns the `openssl.ocsp.response` associated with the `ssl` object (or *nil* if one has not been set).

3.1.16 `openssl.digest`

Binds the “EVP_MD_CTX” OpenSSL object, which represents a cryptographic message digest (i.e. hashing) algorithm instance.

`digest.interpose(name, function)`

Add or interpose a digest class method. Returns the previous method, if any.

`digest.new([type])`

Return a new digest instance using the specified algorithm *type*. *type* is a string suitable for passing to the OpenSSL routine `EVP_get_digestbyname`, and defaults to “sha1”.

`digest:update([string [, ...]])`

Update the digest with the specified string(s). Returns the digest object.

`digest:final([string [, ...]])`

Update the digest with the specified string(s). Returns the final message digest as a binary string.

3.1.17 `openssl.hmac`

Binds the “HMAC_CTX” OpenSSL object, which represents a cryptographic HMAC algorithm instance.

`hmac.interpose(name, function)`

Add or interpose an HMAC class method. Returns the previous method, if any.

`hmac.new(key [, type])`

Return a new HMAC instance using the specified *key* and *type*. *key* is the secret used for HMAC authentication. *type* is a string suitable for passing to the OpenSSL routine `EVP_get_digestbyname`, and defaults to “sha1”.

`hmac:update([string [, ...]])`

Update the HMAC with the specified string(s). Returns the HMAC object.

`hmac:final([string [, ...]])`

Update the HMAC with the specified string(s). Returns the final HMAC checksum as a binary string.

3.1.18 openssl.cipher

Binds the “EVP_CIPHER_CTX” OpenSSL object, which represents a cryptographic cipher instance.

`cipher.interpose(name, function)`

Add or interpose a cipher class method. Returns the previous method, if any.

`cipher.new(type)`

Return a new, uninitialized cipher instance. *type* is a string suitable for passing to the OpenSSL routine `EVP_get_cipherbyname`, typically of a form similar to “AES-128-CBC”.

The cipher is uninitialized because some algorithms support or require additional *ad hoc* parameters before key initialization. The API still allows one-shot encryption like “`cipher.new(type):encrypt(key, iv):final(plaintext)`”.

`cipher:encrypt(key [, iv] [, padding] [, aad])`

Initialize the cipher in encryption mode. *key* and *iv* are binary strings with lengths equal to that required by the cipher instance as configured. In other words, key stretching and other transformations must be done explicitly. If the mode does not take an IV or equivalent, such as in ECB mode, then it may be nil. *padding* is a boolean which controls whether PKCS padding is applied, and defaults to true. *aad* is optional additional authenticated data. Returns the cipher instance.

`cipher:decrypt(key [, iv] [, padding])`

Initialize the cipher in decryption mode. *key*, *iv*, and *padding* are as described in `:encrypt`. Returns the cipher instance.

`cipher:update([string [, ...]])`

Update the cipher instance with the specified string(s). Returns a string on success, or nil and an error message on failure. The returned string may be empty if no blocks can be flushed.

`cipher:final([string [, ...]])`

Update the cipher with the specified string(s). Returns the final output string on success, or nil and an error message on failure. The returned string may be empty if all blocks have already been flushed in prior `:update` calls.

`cipher:getTag(len)`

Returns the authentication tag for the ciphertext (GCM ciphers only) as a binary string. This method can only be called when encrypting data, and must be called after all data has been processed (i.e. after calling `:final()`).

`cipher:setTag(tag)`

Sets the provided binary string as the expected authentication tag for the forthcoming ciphertext (GCM ciphers only). This method can only be called when decrypting data, and must be called before `:final()` to ensure the ciphertext integrity can be verified successfully.

3.1.19 openssl.ocsp.response

Binds OpenSSL's OCSP_RESPONSE object.

`response:getBasic()`

Returns a `openssl.ocsp.basic` representation of the object contained within the OCSP response.

`response:toString()`

Returns a human readable description of the OCSP response as a string.

`response:toPEM()`

Returns the OCSP response as a PEM encoded string.

3.1.20 openssl.ocsp.basic

Binds OpenSSL's OCSP_BASICRESP object.

`basic:verify([certs [, store[, flags]])`

Verifies that the OCSP response is signed by a certificate in the `openssl.x509.chain certs` or a trusted certificate in `openssl.x509.store store`.

3.1.21 openssl.rand

Binds OpenSSL’s random number interfaces.

OpenSSL will automatically attempt to seed itself from the system. The only time this could theoretically fail is if `/dev/urandom` (or similar) were not visible or could not be opened. This might happen if within a chroot jail, or if a file descriptor limit were reached.

`rand.bytes(count)`

Returns *count* cryptographically-strong bytes as a single string. Throws an error if OpenSSL could not complete the request—e.g. because the CSPRNG could not be seeded.

`rand.ready()`

Returns a boolean describing whether the CSPRNG has been properly seeded.

In the default CSPRNG engine this routine will also attempt to seed the system if not already. Because seeding only needs to happen once per process to ensure a successful `RAND.bytes` invocation⁶, it may be prudent to assert on `rand:ready()` at application startup.

`rand.uniform([n])`

Returns a cryptographically strong uniform random integer in the interval $[0, n - 1]$. If *n* is omitted, the interval is $[0, 2^{64} - 1]$.

The routine operates internally on 64-bit unsigned integers.⁷ Because neither Lua 5.1 nor 5.2 support 64-bit integers, it’s probably best to generate numbers that fit the integral range of your Lua implementation. Lua 5.3 supports a new arithmetic type for 64-bit signed integers in two’s-complement representation. This new arithmetic type will be used for argument and return values when available.

3.1.22 openssl.des

Binds OpenSSL’s DES interfaces. These bindings are incomplete. No modern protocol would ever need to use these directly. However, legacy protocols like Windows LAN Manager authentication require some of these low-level interfaces.

`des.string_to_key(password)`

Converts an arbitrary length string, *password*, to a DES key using `DES_string_to_key`. Returns an 8-byte string.

Note that OpenSSL’s `DES_string_to_key` is not compatible with Windows LAN Manager hashing scheme. Use `des.set_odd_parity` instead. See `examples/lm.hash`.

⁶At least this appeared to be the case when examining the source code of OpenSSL 1.0.1. See `md_rand.c` near line 407—“Once we’ve had enough initial seeding we don’t bother to adjust the entropy count, though, because we’re not ambitious to provide *information-theoretic* randomness.”

⁷Actually, `unsigned long long`.

`des.set_odd_parity(key)`

Applies `DES_set_odd_parity` to the string *key*. Only the first 8 bytes of *key* are processed. Returns an 8-byte string.

3.1.23 openssl.kdf

Binds OpenSSL's Key Derivation Function interfaces.

`kdf.derive(options)`

Derive a key given the table of *options*, different KDF types require different options. Accepted options are:

field	type	description
.type	string	key derivation algorithm—"PBKDF2", "id-scrypt", "hkdf", "TLS-PRF1" or other depending on your version of OpenSSL
.outlen	number	the desired output size
.pass	string	password
.salt	string	salt
.iter	number	iteration count
.md	string	digest to use
.key	string	key
.maxmem_bytes	number	amount of RAM key derivation may maximally use (in bytes)
.secret	string	TLS1-PRF secret
.seed	string	TLS1-PRF seed
.hkdf_mode	string	the HKDF mode to use, one of "extract_and_expand", "extract_only" or "expand_only"
.info	string	HKDF info value
.N	number	scrypt "N" parameter to use
.r	number	scrypt "r" parameter to use
.p	number	scrypt "p" parameter to use

4 Examples

These examples and others are made available under examples/ in the source tree.

4.1 Self-Signed Certificate

```
1  --
  -- Example self-signed X.509 certificate generation.
3  --
  -- Skips intermediate CSR object, which is just an antiquated way for
5  -- specifying subject DN and public key to CAs. See API documentation for
  -- CSR generation.
7  --
  local pkey = require"openssl.pkey"
9  local x509 = require"openssl.x509"
  local name = require"openssl.x509.name"
11 local altname = require"openssl.x509.altname"

13 -- generate our public/private key pair
  local key = pkey.new{ type = "EC", curve = "prime192v1" }
15
  -- our Subject and Issuer DN (self-signed, so same)
17 local dn = name.new()
  dn:add("C", "US")
19 dn:add("ST", "California")
  dn:add("L", "San Francisco")
21 dn:add("O", "Acme, Inc")
  dn:add("CN", "acme.inc")
23
  -- our Alternative Names
25 local alt = altname.new()
  alt:add("DNS", "acme.inc")
27 alt:add("DNS", "*.acme.inc")

29 -- build our certificate
  local crt = x509.new()
31
  crt:setVersion(3)
33 crt:setSerial(42)

35 crt:setSubject(dn)
  crt:setIssuer(crt:getSubject())
37 crt:setSubjectAlt(alt)

39 local issued, expires = crt:getLifetime()
  crt:setLifetime(issued, expires + 60) -- good for 60 seconds
41
  crt:setBasicConstraints{ CA = true, pathLen = 2 }
```

```
43 crt:setBasicConstraintsCritical(true)

45 crt:setPublicKey(key)
   crt:sign(key)
47
   -- pretty-print using openssl command-line utility.
49 io.popen("openssl_x509_text_noout", "w"):write(tostring(crt))
```

4.2 Signature Generation & Verification

```
1  --
  -- Example public-key signature verification.
3  --
  local pkey = require"openssl.pkey"
5  local digest = require"openssl.digest"

7  -- generate a public/private key pair
  local key = pkey.new{ type = "EC", curve = "prime192v1" }
9
  -- digest our message using an appropriate digest
11 local data = digest.new "sha1"
   data:update(... or "hello_world")
13
  -- generate a signature for our data
15 local sig = key:sign(data)

17 -- to prove verification works, instantiate a new object holding just
  -- the public key
19 local pub = pkey.new(key:toPEM"public")

21 -- a utility routine to output our signature
  local function tohex(b)
23     local x = ""
     for i = 1, #b do
25         x = x .. string.format("%.2x", string.byte(b, i))
     end
27     return x
  end
  end
29
  print("okay", pub:verify(sig, data))
31 print("type", pub:type())
   print("sig", tohex(sig))
```